

UFMA – ABI Computação – Estrutura de Dados I - Segunda Prova 2

Questão 1: Escreva um algoritmo que recebe uma lista linear simplesmente encadeada L e um inteiro k, e remove o nó imediatamente seguinte ao k-ésimo nó da lista:

Você receberá uma lista simplesmente encadeada e um número k.

Sua tarefa: remover o nó que vem **LOGO APÓS** o k-ésimo nó da lista.

Exemplo: se $k = 3$, remova o 4º nó (o nó que está depois do 3º).

`void removeK(Sllist *L, int k);`

Questão 2: Escreva um algoritmo `copialInvertida(L1, L2)` para montar uma cópia L2 da lista L1. Ambas as listas são duplamente encadeadas e LINEARES (não são circulares). Os elementos de L2 devem aparecer na ordem INVERSA de L1.

Você receberá UMA lista duplamente encadeada L1.

Você deve criar uma CÓPIA INVERTIDA dessa lista chamada L2.

DETALHE IMPORTANTE: os elementos de L2 devem estar na ORDEM INVERSA em relação a L1.

Exemplo: se $L1 = [10, 20, 30]$, então $L2 = [30, 20, 10]$.

`Dllist *copia(Dllist *L1);`

Você **PODE** alocar novos nós nesta questão (novos struct DNo), pois é uma cópia.

Questão 3: Escreva um algoritmo `CombinaOrdenadas(L1, L2)` que recebe duas listas lineares duplamente encadeadas já ordenadas e retorna uma única lista duplamente encadeada ordenada contendo todos os nós das duas listas.

Você receberá DUAS listas duplamente encadeadas ORDENADAS: L1 e L2.

Sua tarefa: combinar as duas em UMA ÚNICA lista duplamente encadeada ordenada.

RESTRIÇÃO IMPORTANTE: **NÃO PODE** alocar novos nós. Reutilize os nós existentes.

Exemplo: $L1 = [1, 3, 5]$ e $L2 = [2, 4, 6] \Rightarrow$ resultado = $[1, 2, 3, 4, 5, 6]$.

Função de comparação cmp funciona assim:

`cmp(a, b)` retorna -1 se $a < b$ (a vem antes de b na ordem)

`cmp(a, b)` retorna 0 se $a == b$ (são iguais)

`cmp(a, b)` retorna +1 se $a > b$ (a vem depois de b na ordem)

`Dllist *CombinaOrdenadas(Dllist *L1, Dllist *L2, int (*cmp)(void *, void *));`

DICA: Compare o primeiro de l1 com o primeiro de l2 e mova para a lista 3 o menor dos dois.

Questão 4: Escreva um algoritmo que recebe uma lista circular simplesmente encadeada e conta o número de vezes que um determinado dado, identificado por key, aparece.

Você receberá uma lista **CIRCULAR SIMPLEMENTE ENCADEADA** e uma chave de busca.

Sua tarefa: contar quantas vezes o dado identificado pela chave aparece na lista.

LEMBRETE: em uma lista circular, o último nó aponta de volta para o primeiro nó!

Exemplo: lista = $[5, 3, 5, 8, 5]$ e key = 5 $\Rightarrow$ retorne 3.

A função de comparação cmp funciona assim:

`cmp(key, data)` retorna 1 (TRUE) se 'key' identifica o dado em 'data'

`cmp(key, data)` retorna 0 (FALSE) se 'key' NÃO identifica o dado em 'data'

`int lcseGetNumberOfOccurrences(Sllist *l, void *Keyd, int (*cmp)(void *, void *));`